ASIAN INSTITUTE OF MANAGEMENT

Machine Learning 3

RNN Fundamentals II

Session 11 and 12

MSDS 2026

11 May 2026

ASIAN INSTITUTE OF MANAGEMENT

Class Administrative Matters

Course Schedule

Session 13 and 14 (Thursday, May 14) – RNN Application: NLP

Final Project Presentation (Wednesday June 10)

Session 11 and 12 – RNN Fundamentals II

Gameplan

Session 11

RNN Challenges

Modern RNNs

Session 12

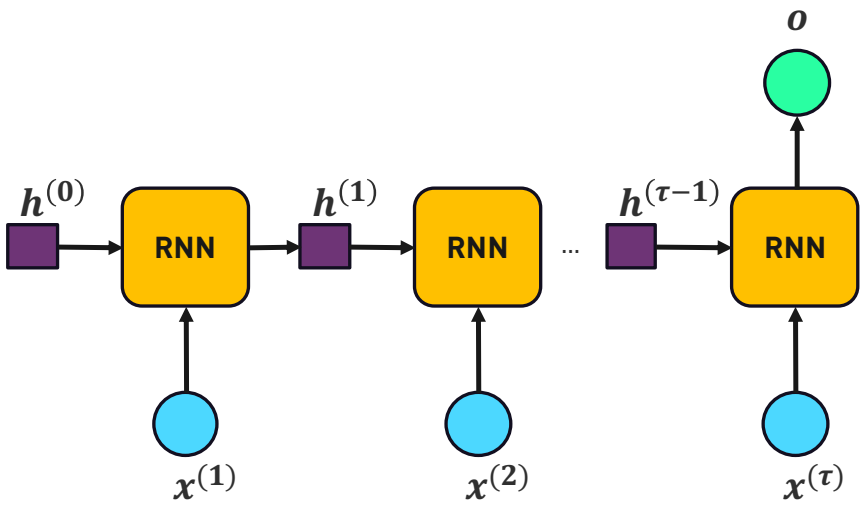
RNN Application: Time Series Forecasting

RNN Challenges

RNN Challenges

Simple RNN Architecture

$$\boldsymbol{h}^{(t)} = g\left(\boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)} + \boldsymbol{b}\right)$$



To train RNNs, we need to perform the ***backpropagation through time (BPTT)*** algorithm

$$\frac{d\boldsymbol{h}^{(t)}}{d\boldsymbol{W}} = \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{W}} + \sum_{i=1}^{t-1}\left(\frac{d\boldsymbol{h}^{(i)}}{d\boldsymbol{W}}\right)\cdot\left(\prod_{j=i+1}^{t}\frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}}\right)$$

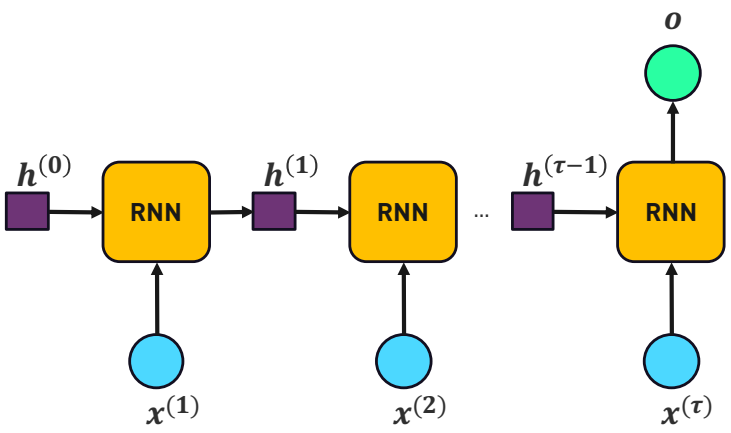
$$\frac{d\boldsymbol{h}^{(t)}}{d\boldsymbol{U}} = \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{U}} + \sum_{i=1}^{t-1}\left(\frac{d\boldsymbol{h}^{(i)}}{d\boldsymbol{U}}\right)\cdot\left(\prod_{j=i+1}^{t}\frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}}\right)$$

RNN Challenges

Challenge of Long-Term Dependencies

Consider,

$$\prod_{j=i+1}^{t} \frac{\partial h^{(j)}}{\partial h^{(j-1)}}$$



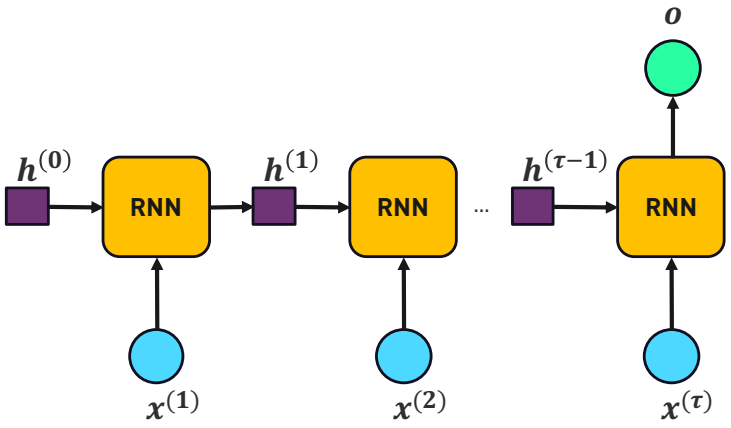
$$h^{(t)} = g\left(Wh^{(t-1)} + Ux^{(t)} + b\right)$$

RNN Challenges

Challenge of Long-Term Dependencies

Consider,

$$\prod_{j=i+1}^{t} \frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}} = \prod_{j=i+1}^{t} diag\left(g'\left(\boldsymbol{W}\boldsymbol{h}^{(j-1)} + \boldsymbol{U}\boldsymbol{x}^{(j)} + \boldsymbol{b}\right)\right)\boldsymbol{W}$$



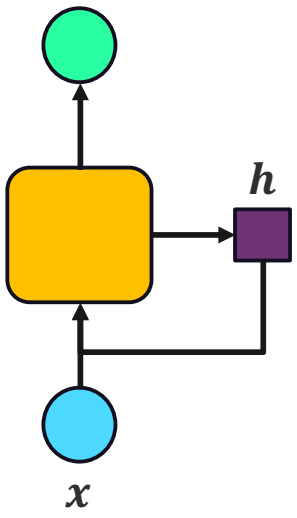
$$\boldsymbol{h}^{(t)} = g\left(\boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)} + \boldsymbol{b}\right)$$

RNN Challenges

Challenge of Long-Term Dependencies

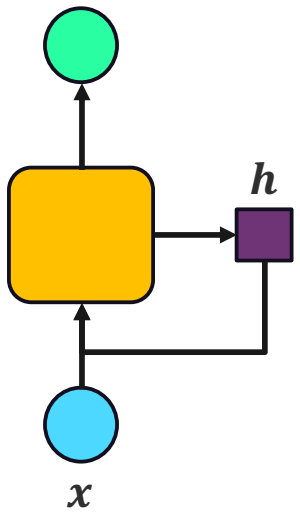
$$\prod_{j=i+1}^{t} \frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}} = \prod_{j=i+1}^{t} diag\left(g'\left(\boldsymbol{W}\boldsymbol{h}^{(j-1)} + \boldsymbol{U}\boldsymbol{x}^{(j)} + \boldsymbol{b}\right)\right)\boldsymbol{W}$$

$\|\boldsymbol{W}\| > \mathbf{1}$



exploding gradient

$\|\boldsymbol{W}\| < \mathbf{1}$



vanishing gradient

RNN Challenges

Challenge of Long-Term Dependencies

Handling RNN Challenges:

- Truncated BPTT
- Gradient Clipping
- Weight Regularization
- Modern RNNs: LSTM and GRU

RNN Challenges

Handling RNN Challenges

Truncated BPTT

In truncated BPTT, we truncate the sum in BPTT after κ steps

BPTT

$$\frac{d\boldsymbol{h}^{(t)}}{d\boldsymbol{W}} = \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{W}} + \sum_{i=1}^{t-1} \left(\frac{d\boldsymbol{h}^{(i)}}{d\boldsymbol{W}} \right) \cdot \left(\prod_{j=i+1}^{t} \frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}} \right)$$

Truncated BPTT

$$\frac{d\boldsymbol{h}^{(t)}}{d\boldsymbol{W}} = \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{W}} + \sum_{i=t-\kappa}^{t-1} \left(\frac{d\boldsymbol{h}^{(i)}}{d\boldsymbol{W}} \right) \cdot \left(\prod_{j=i+1}^{t} \frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}} \right)$$

RNN Challenges

Handling RNN Challenges

Gradient Clipping

Clipping by norm

If $\left\|\frac{dL}{dW}\right\| \geq \text{threshold}$, then set the magnitude of the gradient equal to the threshold.

Clipping by value

If $\left(\frac{dL}{dW}\right)_i \geq \text{threshold}$, then set the value of the gradient element equal to the threshold.

RNN Challenges

Handling RNN Challenges

Weight Regularization

We add a regularizer onto our loss function that to prefer solutions for which the error preserves norm as it travels back in time.

$$\Omega = \sum_k \Omega_k = \sum_k \left(\frac{\left\| \frac{dL}{dh^{(k+1)}} \cdot \frac{dh^{(k+1)}}{dh^{(k)}} \right\|}{\left\| \frac{dL}{dh^{(k+1)}} \right\|} - 1 \right)^2$$

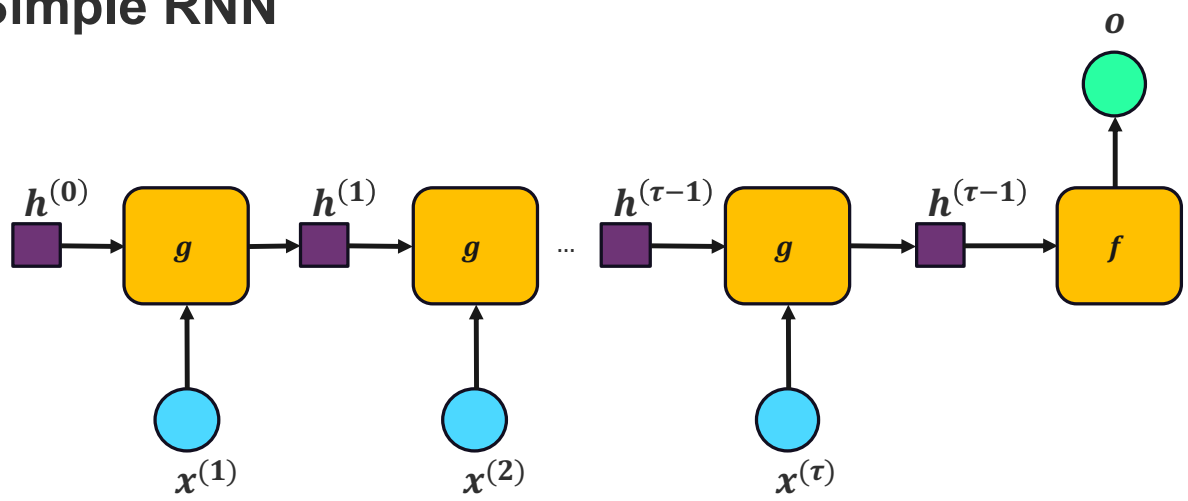
Pascanu, R., Mikolov, T., & Bengio, Y. (2013, May). On the difficulty of training recurrent neural networks.

Modern RNNs

Modern RNNs

Challenge of Long-Term Dependencies

Simple RNN



Forward Pass

$$\boldsymbol{o} = f\left(\boldsymbol{h}^{(\tau)};\ \boldsymbol{\theta}\right)$$

$$\boldsymbol{h}^{(\tau-1)} = g\left(\boldsymbol{h}^{(\tau-2)}, \boldsymbol{x}^{(3)};\ \boldsymbol{\theta}\right)$$

...

$$\boldsymbol{h}^{(2)} = g\left(\boldsymbol{h}^{(1)}, \boldsymbol{x}^{(2)};\ \boldsymbol{\theta}\right)$$

$$\boldsymbol{h}^{(1)} = g\left(\boldsymbol{h}^{(0)}, \boldsymbol{x}^{(1)};\ \boldsymbol{\theta}\right)$$

Backpropagation

$$\frac{d\boldsymbol{h}^{(\tau)}}{d\boldsymbol{\theta}} = \frac{\partial \boldsymbol{h}^{(\tau)}}{\partial \boldsymbol{\theta}} + \sum_{i=1}^{\tau-1}\left(\frac{d\boldsymbol{h}^{(i)}}{d\boldsymbol{\theta}}\right)\cdot\left(\prod_{j=i+1}^{t}\frac{\partial \boldsymbol{h}^{(j)}}{\partial \boldsymbol{h}^{(j-1)}}\right)$$

Modern RNNs

Challenge of Long-Term Dependencies

Forward Pass

$$\boldsymbol{o} = f(\boldsymbol{h_n};\ \boldsymbol{\phi})$$

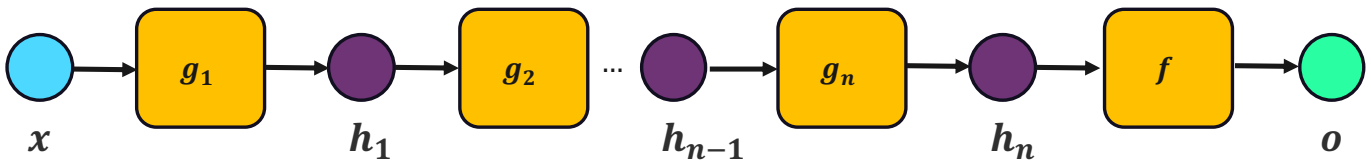
$$\boldsymbol{h_n} = g_n(\boldsymbol{h_{n-1}};\ \boldsymbol{\theta_n})$$

$$\ldots$$

$$\boldsymbol{h_2} = g_2(\boldsymbol{h_1};\ \boldsymbol{\theta_2})$$

$$\boldsymbol{h_1} = g_1(\boldsymbol{x};\ \boldsymbol{\theta_1})$$

Deep NN



Backpropagation (layer 1)

$$\frac{dL}{d\theta_1} = \frac{dL}{do} \cdot \frac{do}{dh_n} \ldots \cdot \frac{dh_2}{dh_1} \cdot \frac{dh_1}{d\theta_1}$$

Modern RNNs

Challenge of Long-Term Dependencies

Forward Pass

$$\boldsymbol{o} = f(\boldsymbol{h_n};\ \boldsymbol{\phi})$$

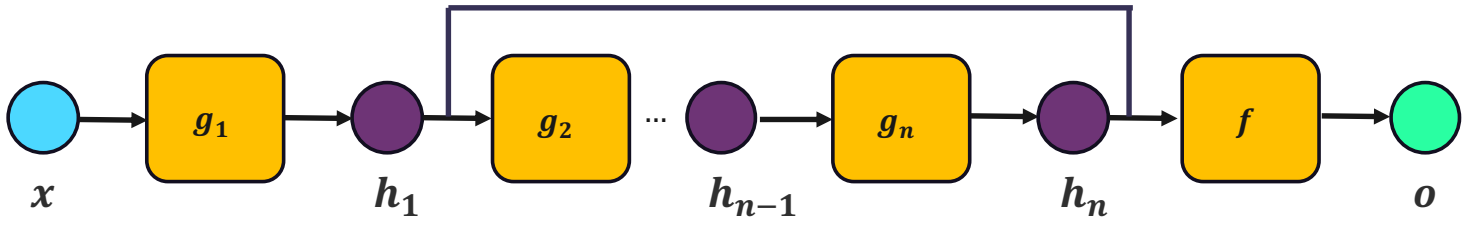
$$\boldsymbol{h_n} = \boldsymbol{h_1} + g_n(\boldsymbol{h_{n-1}};\ \boldsymbol{\theta_n})$$

...

$$\boldsymbol{h_2} = g_2(\boldsymbol{h_1};\ \boldsymbol{\theta_2})$$

$$\boldsymbol{h_1} = g_1(\boldsymbol{x};\ \boldsymbol{\theta_1})$$

Residual NN



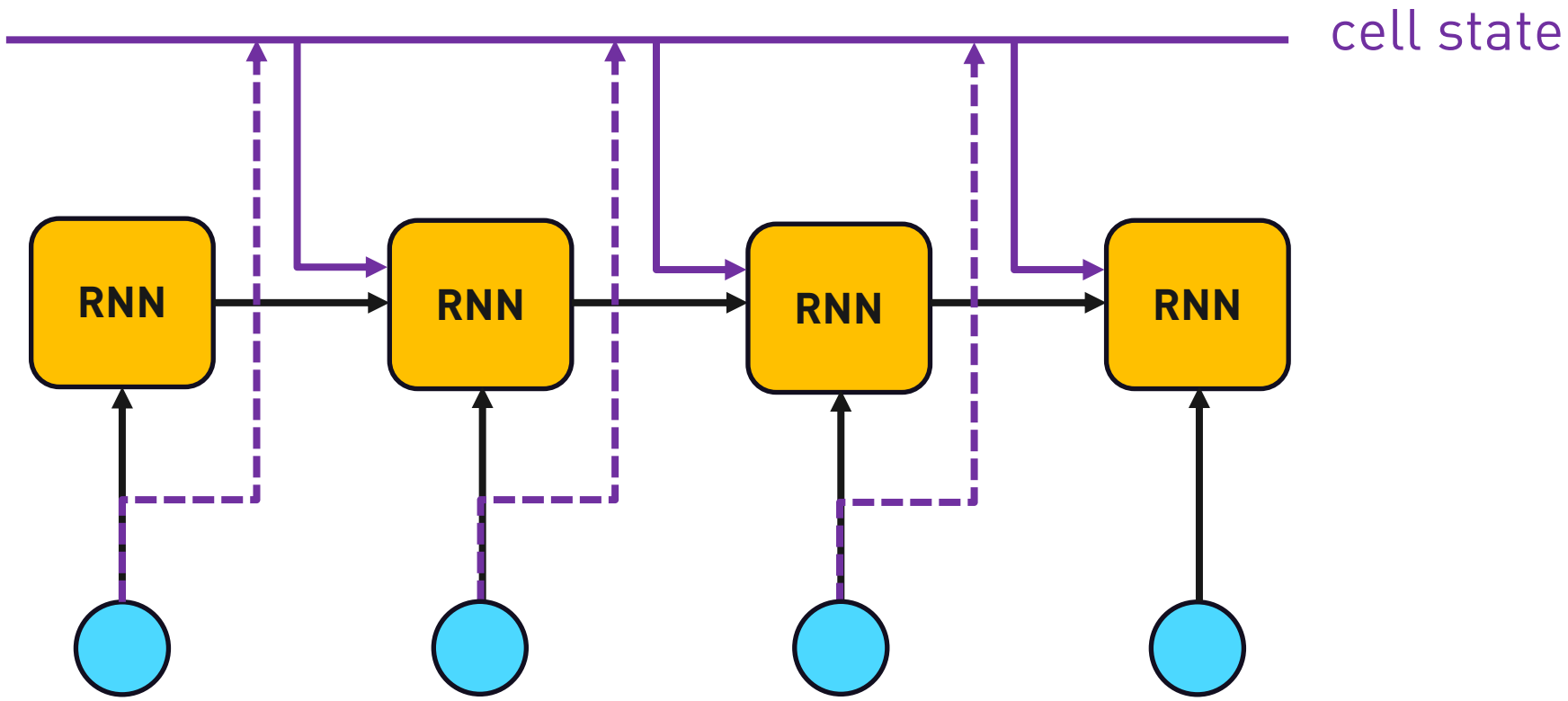
Backpropagation (layer 1)

$$\frac{dL}{d\theta_1} = \frac{dL}{do} \cdot \frac{do}{dh_n} \cdot \frac{dh_n}{d\theta_1}$$

$$\frac{dh_n}{d\theta_1} = \frac{dh_1}{d\theta_1} + \frac{dg_n}{dh_{n-1}} \cdot \frac{dh_{n-1}}{dh_{n-2}} \cdots \cdot \frac{dh_2}{dh_1} \cdot \frac{dh_1}{d\theta_1}$$

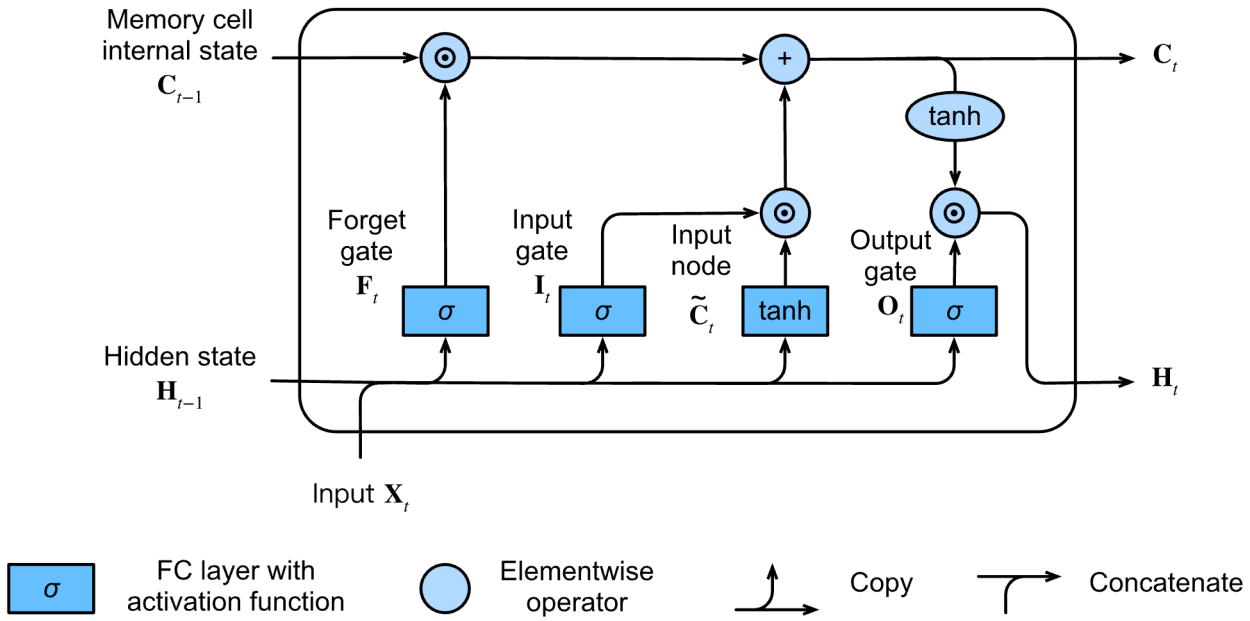
Modern RNNs

LSTM Architecture



Modern RNNs

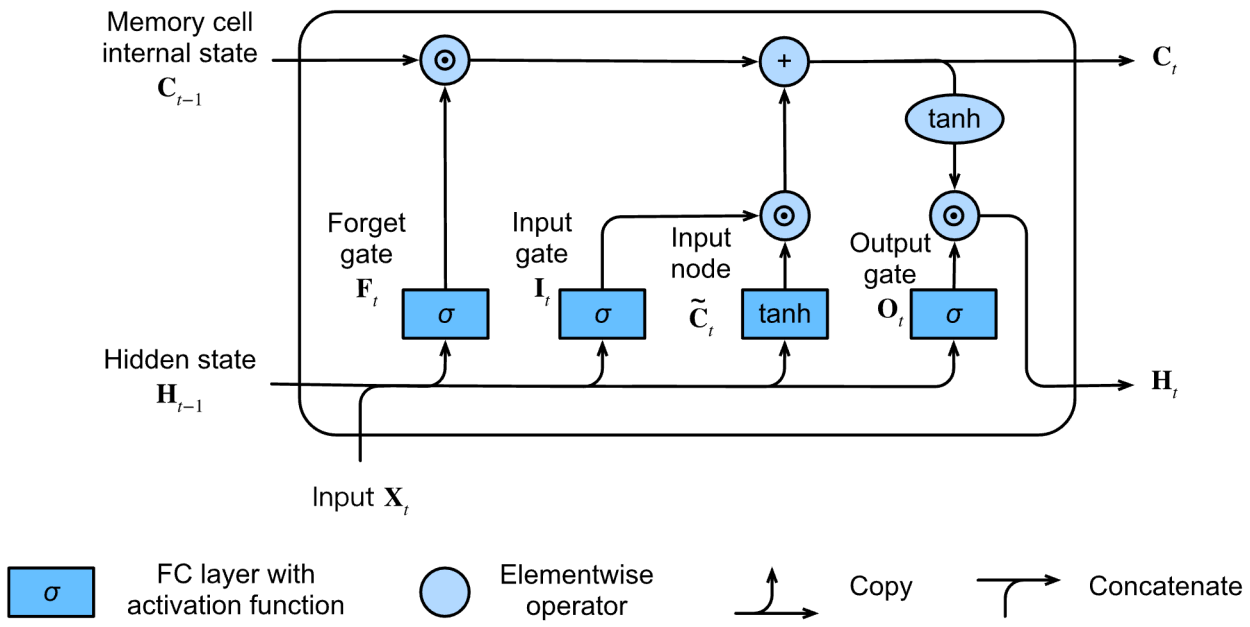
LSTM Architecture



Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.

Modern RNNs

LSTM Architecture



LSTM Gates

Input Gate – determines how much of the input should be included to the cell state

Forget Gate – determines whether to keep current value of the cell state

Output Gate – determines how much influence the cell state has on the output

Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.

LSTM Architecture

LSTM Gates

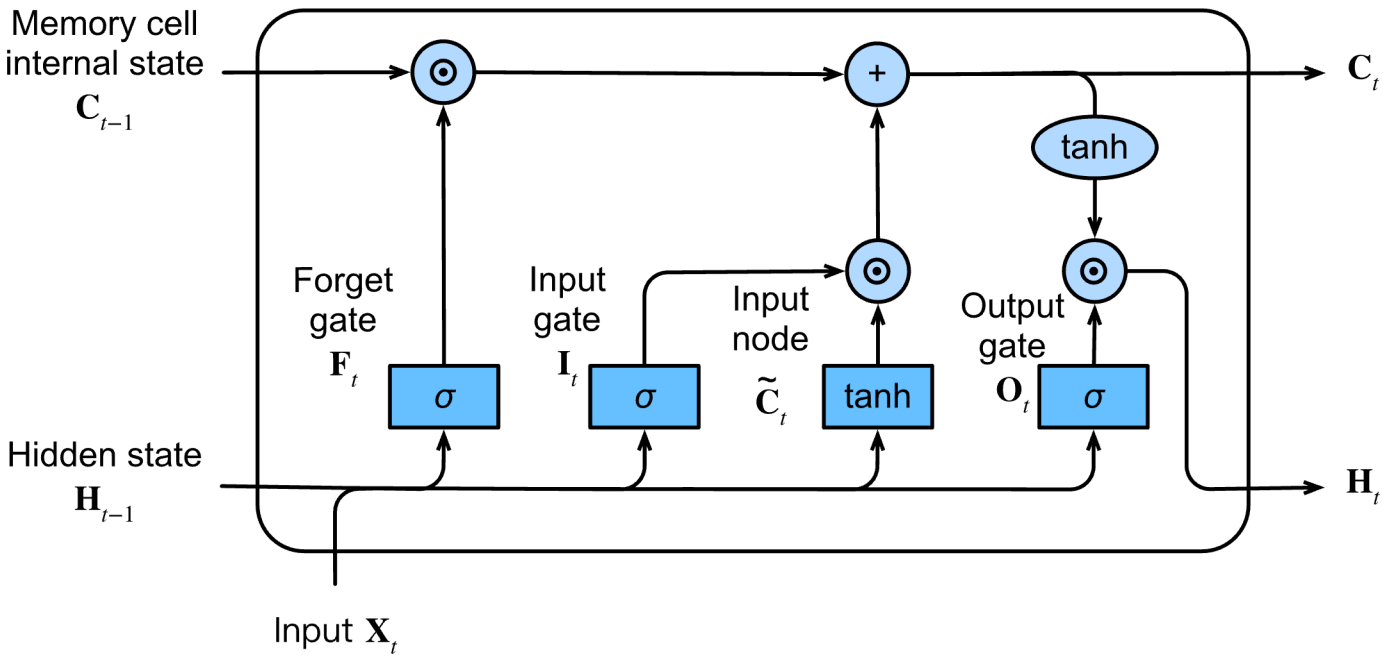
$$I^{(t)} = \sigma\left(U_I x^{(t)} + W_I h^{(t-1)} + b_I\right)$$

$$F^{(t)} = \sigma\left(U_F x^{(t)} + W_F h^{(t-1)} + b_F\right)$$

$$O^{(t)} = \sigma\left(U_O x^{(t)} + W_O h^{(t-1)} + b_O\right)$$

Candidate Value

$$\tilde{C}^{(t)} = \tanh\left(U_C x^{(t)} + W_C h^{(t-1)} + b_C\right)$$



Memory Cell Update

$$C^{(t)} = F^{(t)} \cdot C^{(t-1)} + I^{(t)} \cdot \tilde{C}^{(t)}$$

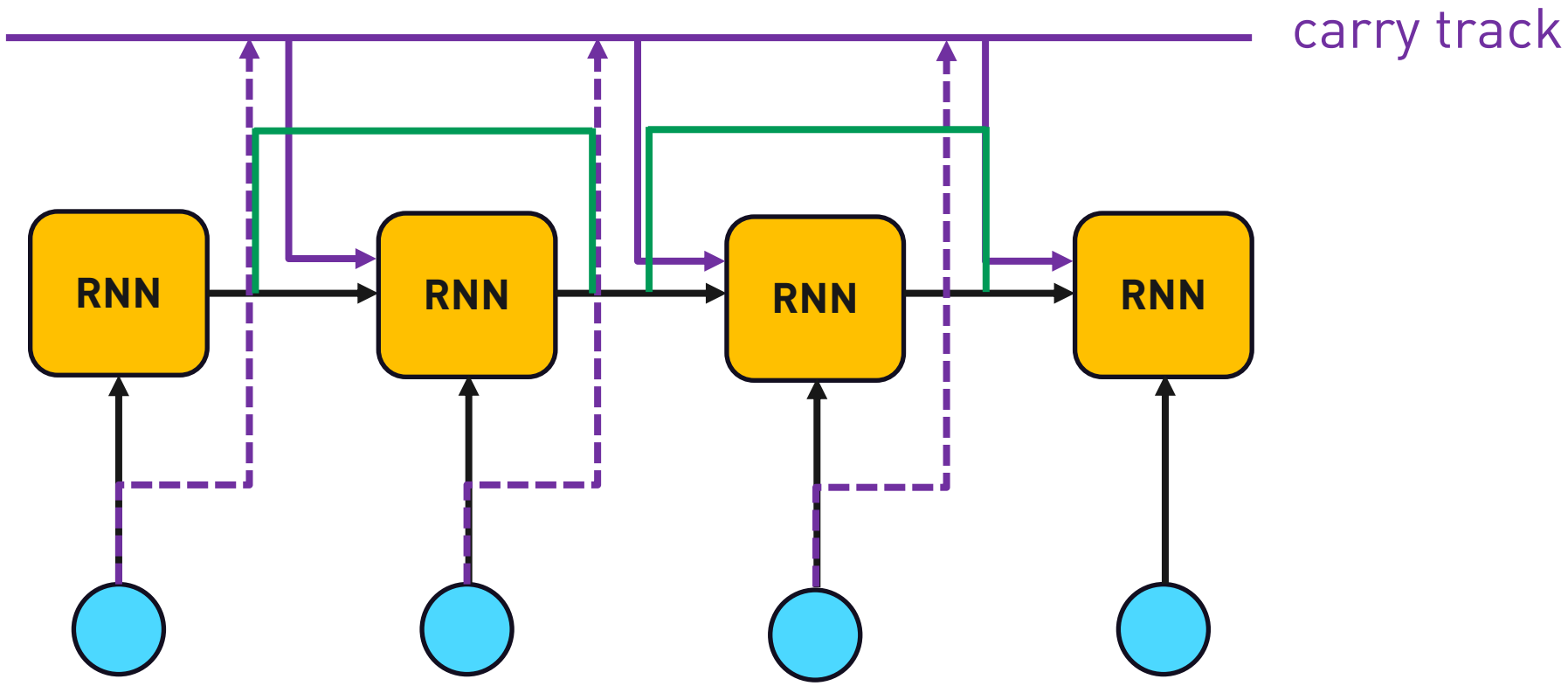
Hidden State Update

$$h^{(t)} = O^{(t)} \cdot \tanh\left(C^{(t)}\right)$$

Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.

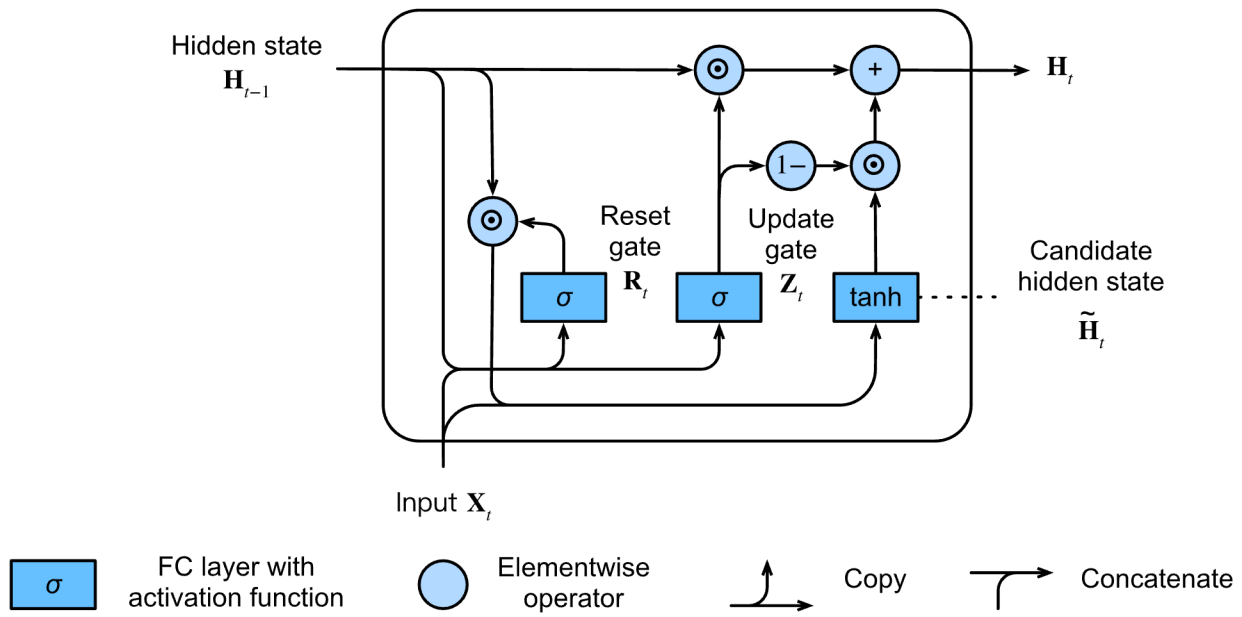
Modern RNNs

GRU Architecture



Modern RNNs

GRU Architecture



GRU Gates

Reset Gate – controls how much of the previous state should be remembered

Update Gate – controls how much of the old state influences the new state

Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.

GRU Architecture

GRU Gates

$$R^{(t)} = \sigma\left(U_R x^{(t)} + W_R h^{(t-1)} + b_R\right)$$

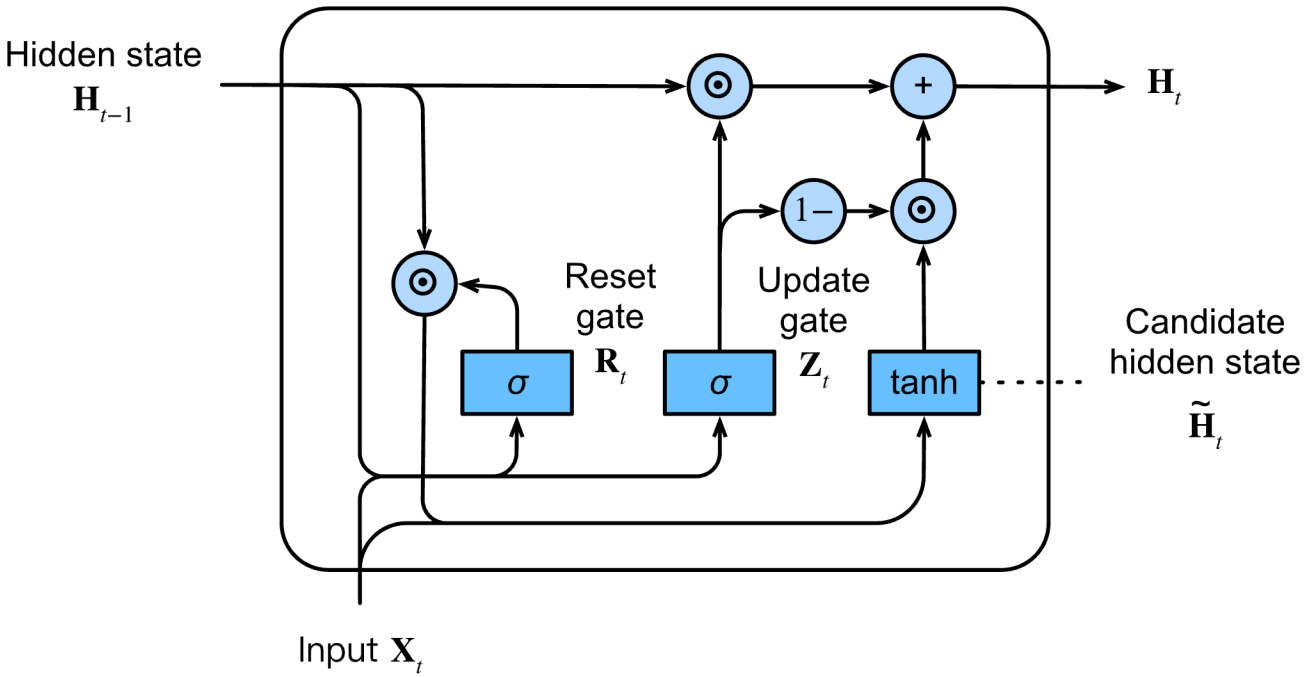
$$Z^{(t)} = \sigma\left(U_Z x^{(t)} + W_Z h^{(t-1)} + b_Z\right)$$

Candidate Hidden State

$$\tilde{h}^{(t)} = \tanh\left(U_C x^{(t)} + W_C\left(R^{(t)} \cdot h^{(t-1)}\right) + b_C\right)$$

Hidden State Update

$$h^{(t)} = Z^{(t)} \cdot h^{(t-1)} + \left(1 - Z^{(t)}\right) \cdot \tilde{h}^{(t)}$$



Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.

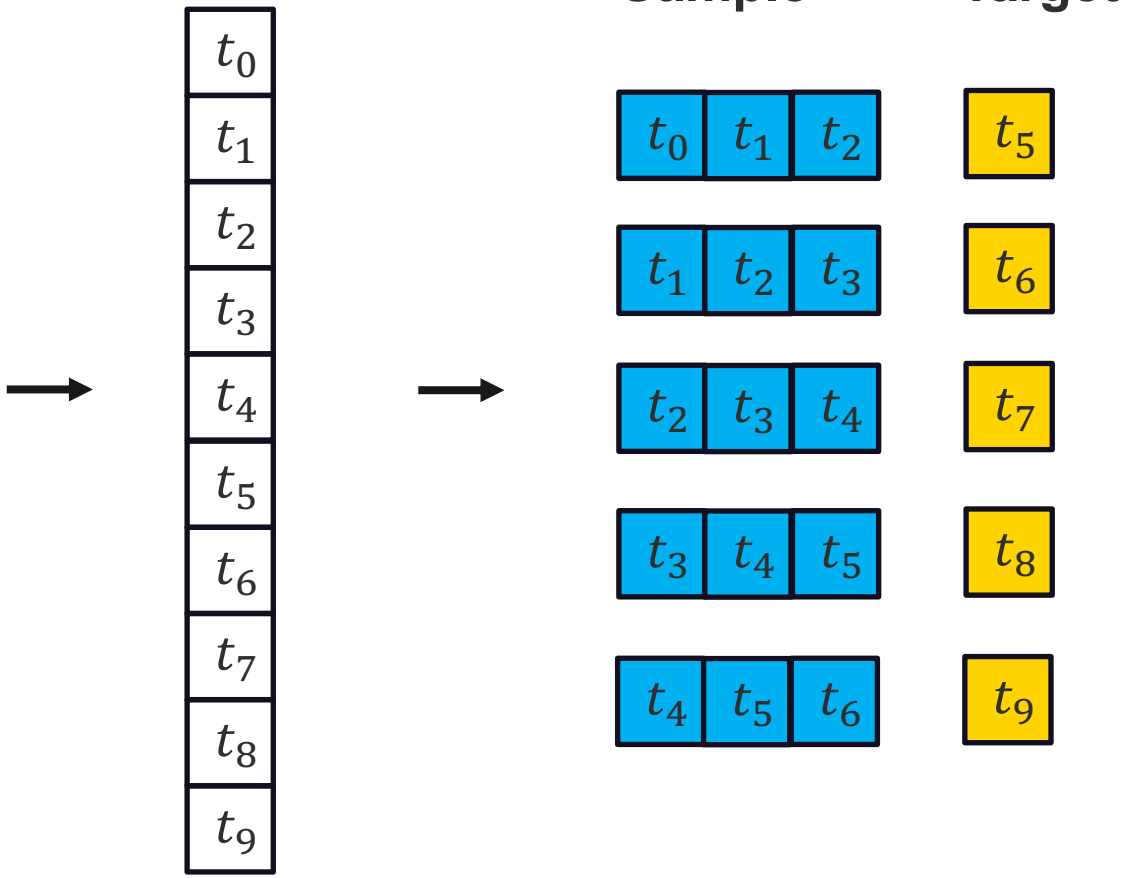
Time Series Forecasting using RNNs

Timeseries Dataset

Generating the sample-target pairs for timeseries

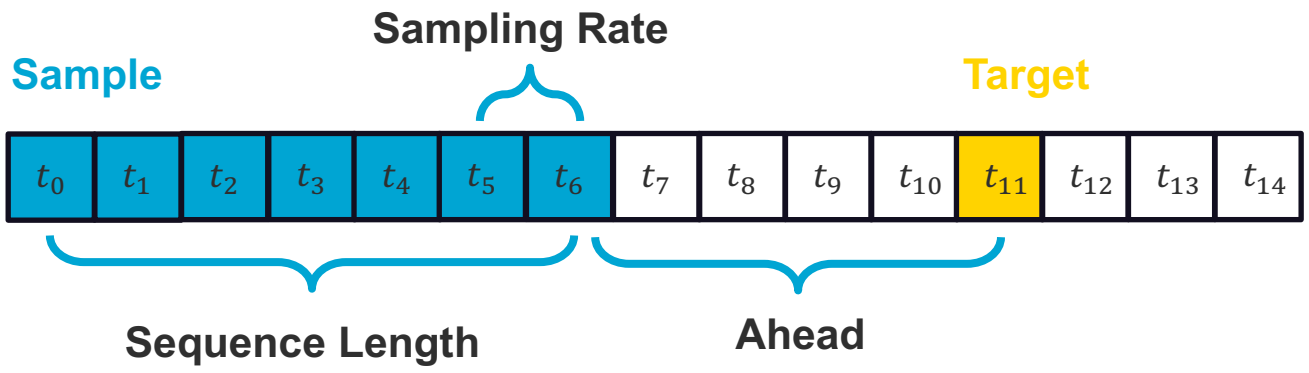
Timeseries Data

Date	Daily Sales
2020-04-01	171
2020-04-02	180
2020-04-03	212
2020-04-04	234
2020-04-05	239
2020-04-06	189
2020-04-07	184
2020-04-08	172
2020-04-09	179
2020-04-10	211



Timeseries Dataset

Timeseries Data Parameters



`timeseries_dataset_from_array` **function**

```
tf_keras.utils.timeseries_dataset_from_array(
    data,
    targets,
    sequence_length,
    sequence_stride=1,
    sampling_rate=1,
    batch_size=128,
    shuffle=False,
    seed=None,
    start_index=None,
    end_index=None,
)
```

Ahead – How far into the future the target is wrt sample

Sequence Length – Length of the sample used as predictor

Sampling Rate – Time step between values within the sample

Sequence Stride – Time difference between consecutive samples